

X-OPM: Explainable Automatic Modeling Methodology for Building Cycle-Accurate Digital On-Chip Power Meters with Reduced Hardware Implementation Overhead

Charles Jiang

Abstract—Proactive power management systems rely on digital on-chip power meters (OPMs) to predict processor power from runtime micro-architectural events. However, most prior OPM modeling methodologies are black-box: the relationship between the selected features and the circuit’s dynamic power cannot be explained, making the resulting models unstable across benchmarks and risky to deploy after tape-out. This paper introduces X-OPM, an explainable automatic modeling methodology for building cycle-accurate digital OPMs. We first propose a fully explainable white-box modeling method, grounded in the low-power design principles of synchronous digital VLSI circuits, that quantitatively relates micro-architectural events to power fluctuation and enables fine-grained trade-offs between model accuracy and hardware cost. We then automate the methodology with a three-step flow—proxy pruning, feature interaction, and model fitting—built on explainable algorithms, and a Generalized Additive Model (GAM) formulation that iteratively decomposes the design into sub-modules. A human-in-the-loop procedure controls the accuracy/area trade-off during training. Evaluated on a commercial, silicon-proven NPU IP with power traces from commercial EDA tools, our methodology achieves over 0.95 R^2 across benchmarks while retaining accuracy on unseen benchmarks, outperforming APOLLO and DEEP, and reaches accuracy comparable to MLP-based OPMs at a significantly lower area overhead.

Index Terms—On-chip power meter, power modeling, proactive power management, explainable machine learning, generalized additive model, di/dt mitigation.

I. INTRODUCTION

THE computation throughput and power consumption of cloud AI processors, including NPUs and GPGPUs, have increased drastically in recent years [1], [2], as shown in Fig. 1. [TODO: briefly explain what the figure says]

Large power consumption causes many issues, such as the di/dt issue, where rapid current transients interacting with the package and power-delivery-network impedance induce supply voltage droops that threaten timing correctness [3]–[8], and the over-current issue: when the maximum power required to perform a computation exceeds the EDC (Electrical Design Current) limit for a sufficiently long time, the processor may break down [9], [10]. [TODO: explain overcurrent more precisely]

To address these challenges, proactive power management systems [10]–[16] have been built to further boost system

performance. A proactive power management system uses a digital on-chip power meter (OPM) to predict the power of the processor in the near future based on runtime micro-architectural events. Typically, the prediction can be made from several nanoseconds to significantly earlier before the actual power change occurs. If the processor detects that a power crisis event, such as a di/dt droop, will happen, it proactively reacts to mitigate the impact of this event.

The accuracy, stability, and implementation overhead of the OPM are the keys to building an effective proactive power management system. Many previous works aim to improve the quality of OPMs. Their approaches can be summarized into the following categories: (1) using stronger models: table-based [17], linear models [18], [19], tree-based/ensemble-tree/GBDT models [20], machine-learning-based models such as MLPs [21]–[23], and feature embedding [24]; (2) selecting better micro-architectural events as features, e.g., PRIMAL [25] and PROPHET [26], [27]; (3) black-box or gray-box modeling: APOLLO [28], DEEP [29], and PANDA [30] proposed to automatically select features from a superset, such as all the nets of the chip, and build a linear model to regress the power [31].

However, when building OPMs in practice, we found that the modeling methodologies of prior arts have limitations. First, quite a number of works do not have a white-box explanation of why their model can explain the variation of digital VLSI circuit power given the micro-architectural events chosen as features. In practice, it is quite common that a model works in certain scenarios but fails in others, and it is hard to locate the root cause of this kind of failure without a fully explainable modeling methodology. Second, the black-box modeling methodology proposed since APOLLO [28] has a deficiency: it does not have a feature interaction mechanism. The micro-architectural events selected by their feature selection algorithms are not guaranteed to have a linear relationship with power. Thus the resulting model sometimes appears to be unstable—its prediction accuracy is not consistent across different benchmarks.

Besides modeling accuracy and stability, three implementation issues affect the modeling strategy of an OPM. (1) The area and power overhead of the OPM must be negligible compared to the processor it models, preferably below 0.1%; thus complex ensemble or machine learning models may not be a feasible choice. (2) The amount of data that can be

collected to train a model is normally limited. Typically, the circuit of the on-chip power meter needs to be developed together with the processor before tape-out. Power can only be collected through expensive netlist simulation (e.g., PTPX) or emulation (e.g., ZeBu) during the processor RTL development procedure, which normally lasts several months. Only a limited number of processor top-level test cases can be collected in such a short period, and it is hard to train a large high-dimensional model given limited data. (3) After tape-out, it is impossible to change the feature selection or the architecture of the power model. If the model built during the RTL development period cannot be fully explained, the risk is high that it may fail in scenarios it has not seen in its training set collected during the RTL development phase.

To solve the above issues, we introduce a new explainable OPM modeling methodology in two steps. We first propose a fully explainable modeling methodology, the X-OPM method, that is capable of quantitatively explaining the relationship between the micro-architectural events (features) we build and the fluctuation of the processor's power. The fully explainable characteristic of the model enables a fine-grained trade-off between model accuracy and the number of selected features, by hand-calculating the power model's output and comparing it with the EDA-simulated power to obtain the modeling error of the test cases. With the ability to quantitatively calculate the modeling error and perform root-cause analysis from the circuit design's perspective, one can be more confident that the constructed features will not lose efficacy after the chip comes back, even though the behavior of pre-silicon power simulation and post-silicon measurement can differ.

In the second step, we propose an automatic modeling algorithm based on the X-OPM modeling method. We propose a three-step approach: proxy pruning, feature interaction, and model fitting. Each step supports choosing from different algorithms, and we deliberately choose explainable ones. For example, GBDT and the attention mechanism can be chosen for the feature interaction step, while a linear model or an MLP (multi-layer perceptron) can be chosen for the regression model. To further improve modeling accuracy and stability, we propose a GAM (Generalized Additive Model) [32] approach: we iteratively split the design into sub-modules, fit them with individual models, and sum the modules up to construct the final processor power model. The iteration stops when R^2 reaches a threshold, e.g., 0.95. Although the GAM-based power model is built automatically in a black-box fashion, its modeling methodology is aligned with the white-box X-OPM method and can thus be fully explained.

Since a black-box method can always go wild if not controlled properly, to retain the fine-grained trade-off characteristic of X-OPM we propose a human-in-the-loop procedure to control the model accuracy and area overhead by iteratively adding or removing features during model training.

We have also designed parameterized hardware to implement the models and to evaluate the area and power overhead of each modeling methodology. The evaluation is performed with commercial EDA tools.

We show that our automatic power modeling methodology achieves over 0.95 R^2 across different benchmarks while

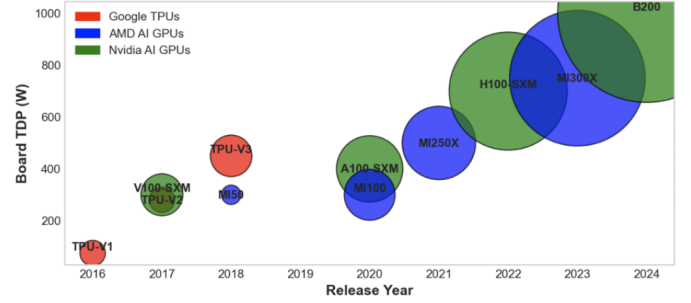


Fig. 1. Board TDP versus release year of recent cloud AI processors (Google TPUs, AMD and Nvidia AI GPUs).

keeping accuracy on unseen benchmarks. The result is better than APOLLO [28] and DEEP [29]. It can also handle a large design on which DEEP cannot finish training. [TODO: experimental results required] Our implementation results show that the approach is $XX\times$ more area efficient than the state-of-the-art proactive power management design reported at ISSCC 2026 [16].

Although recent OPM works favor MLP-based models since they generally have higher accuracy than traditional statistical-learning-based models, we show that MLP-based models exhibit significantly lower area efficiency. Thanks to our proposed modeling method, we achieve accuracy near that of MLPs with significantly lower area overhead.

Our research contributions are as follows.

- We propose a fully explainable modeling methodology, the X-OPM method, that enables fine-grained trade-offs among model accuracy, modeling effort, and hardware overhead.
- We propose a framework to automate the model building procedure of the X-OPM method.
- We evaluate the hardware implementation of different algorithms and confirm that X-OPM based on traditional statistical learning algorithms reaches the accuracy of MLP-based algorithms with much lower hardware overhead.

II. BACKGROUND AND RELATED WORKS

A. On-Chip versus Off-Chip Power Modeling

On-chip power models focus on modeling the dynamic power related to the instructions the processor is currently running, or the current chip status (power- or clock-gated, interrupted, pipeline stalled). In contrast, offline power models normally focus on simulating the power of a given chip, working together with a cycle-accurate performance model as a processor simulator [33], or helping to perform architectural design space exploration (DSE) [30].

Thus, the features favored by on-chip power models normally differ from those of off-chip power models. On-chip features come from on-chip signals, such as internal wires, instruction or event counters, or dedicated PMU event counters—for example, committed instruction counts or AXI-bus write-back counts. Off-chip power models can use other features such as architectural parameters, which gives those models scalability when the architect is considering DSE [30].

This paper focuses only on building the power model. Runtime PVT corner changes, DVFS effects, and the PDN are normally modeled with other algorithms [11], which are outside the scope of this paper.

B. Sources of Prediction Error

Several sources of error lead to unstable prediction:

- **Static error** (>ms): process variation; runtime temperature, voltage, and frequency change; and aging.
- **Measurement error.**
- **Dynamic error** (<ms): poor-quality feature engineering, distribution shift, and poor generalization capability.

Since PTPX simulation takes a long time, data collection may need to continue during development, and there may not be enough data by the time the design is frozen. Fitting an ML model on limited data samples may lead to overfitting, which is hard to detect when the model is black-box. Since the model's input features are hardened after tape-out, it may be too late to discover that the input features are improper after the design has been shipped to the fab. A white-box model, on the other hand, gives confidence because the relationship of the features to the power can be explained to a large extent. Using an explainable model is therefore more important for on-chip machine learning than for off-chip machine learning.

C. Limitations of Current Research

Most existing approaches are black-box, simply applying different machine learning models to the dataset, and lack a methodology for building a scalable, compact, and explainable model. The more black-box the model is, the higher the chance that it overfits the training dataset, i.e., the higher the probability of instability or accuracy drops when facing a new dataset it has not seen before. Although APOLLO [28] discussed how to automatically select features from a superset, its algorithm requires all features in the superset to be linear with the power, which becomes intractable when chip size gets large. DEEP [29] proposed a greedy method, but when the chip size becomes large, it is hard to make it converge.

III. X-OPM METHODOLOGY

We first describe how to build the white-box model, named X-OPM model, and discuss why it works on unseen datasets, and later extend it to an explainable black-box methodology.

A. Dynamic Power Analysis for Synchronous Logic

Fig. 2 shows an example design illustrating the principal mechanisms of how dynamic power is generated in synchronous digital VLSI circuits when the input toggles. Recall that the on-chip power model for a proactive power management system must learn the relationship between power and signal (net) toggling, so the root analysis must be derived from logic design principles. When a synchronous digital VLSI circuit is designed, certain low-power design rules are followed by the circuit designer. These rules can be found not only in processors but in general synchronous digital VLSI circuits, and they form the foundation of the X-OPM model.

We use a deliberately designed toy design to describe these rules. It consists of two pipelined modules—Module A and Module B—whose data flow is controlled by a valid/ready handshake protocol. Module A can be configured to either multiply, or multiply-and-add, by setting the `a_mode` signal low or high. When the handshake signal is low, all registers are clock-gated to hold the values at their Q-ports.

Clock gating and data gating are low-power techniques used extensively in logic design. When the handshake signal is low, the clock of the registers storing the control and data payloads is turned off; thus the dynamic power of the registers, as well as the data toggling of the combinational logic behind the registers, is reduced. In contrast, data gating is used for pure combinational logic: when the mask signal goes low, the input to the adder is masked to zero, so the output of the multiplier does not toggle the combinational logic inside the adder.

Note that in the first cycle of data gating there may still be data toggling. Thus, if the data gating signal toggles at a 50% rate, data gating may totally lose its effectiveness, since every cycle the input data to the combinational circuit is toggling. We regard this as a design bug since a known data gate loses its function; we do not handle this case and accept it as a source of modeling error, since in a normal design this case is rare and should be handled case by case.

We further classify the payload signals of a digital circuit into *control payload* and *data payload*. The control payload comprises the configuration signals controlling the operating mode of the circuit; for example, when `a_mode` is 0, Module A is configured as a multiplier, and when it is 1, as a multiply-and-add. The data payload is the data bus. When clock and data are not gated and the module operates in a given mode, the intensity of the dynamic power of a combinational circuit is determined by the toggle rate of the data.

A *power proxy* is a measurable micro-architectural event, such as the toggling of a signal (net), used to estimate real-time power consumption. From the above description, we summarize four types of power proxies that represent the dynamic power fluctuation when a digital module is stimulated:

- The *busy signal* indicating that the module or part of the module is working on something. The *handshakeA* and *handshakeB* signals in Fig.2. belongs to this proxy type, because when they are asserted, Module A and module B will working – they take the data from the input ports, execute using their combinational circuits, and put the correct result on their output ports.
- The *low power signal*, including the clock gating signal and the data gating signal. The lower power signal can gate some of the circuits power off when type A proxy is 1. e.g., the *handshakeA* signal is the clock gating signal – when it is 0, the register and combinational power of module A is gone; the mask signal is the data gating signal – when it holds zero, the combinational circuit gated by it has no dynamic power.
- The value of the control payload signals. For example, when `a_mode` is 0 only the multiplier's power is present; when it is 1, the power includes both the multiplier and the adder.

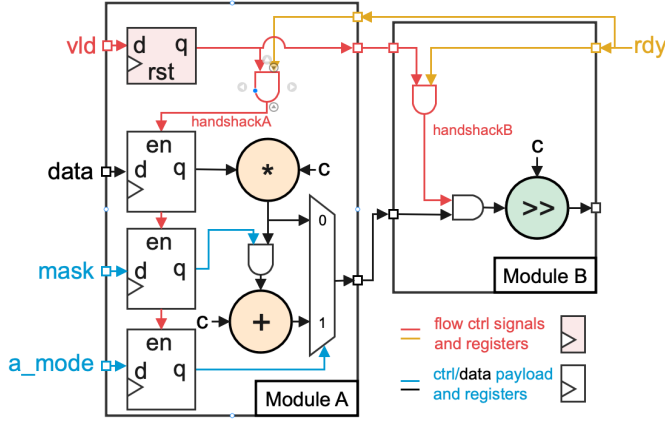


Fig. 2. Illustration of how dynamic power is generated in a typical digital VLSI design (c is a constant).

D. The toggle rate of the data payload signal, typically positively related to the dynamic power of the combinational circuit.

It is worth noting that most other proxy construction principles can also be derived by examining these principles. For example, PROPHET [26] proposes counting the number of zeros in the data bus of the accelerator—more zeros means less computation—which can be considered a realization of proxy type D.

GAM requires all variables affect the result orthogonally. However, in reality, feature interaction exists between the above proposed proxies. One of the most frequently encountered interaction patterns is the multiplicative relationship,

$$\text{type-A} \cdot \text{type-B} \cdot \text{type-D} \quad (1)$$

This multiplicative relationship indicates that type-A,B,D works together to determine the dynamic power. For example, when clock-gated or data-gated, even if the module is working and the input data payload toggles, the circuit will not generate power. Another frequently appearing feature interaction case is the selective relationship,

$$\text{type-C} ? \text{type-D (of Module A)} : \text{type-D (of Module B)}. \quad (2)$$

For example, when $a_mode = 0$, the power of Module A is represented by the data signal of the multiplier, not the adder.

We now move from a single module to the multi-module situation. Synchronous modules typically have multiple pipeline stages; the pipeline stages of the toy design are shown in Fig. 3. We split each module into a sequential and a combinational part, and assume two data items A and B are fed into the pipeline in sequence. In cycle 1, data A goes from the D-port of register R0 to the Q-port of R0, stimulating combinational block 0 to toggle until the signal propagates to register R1. The toggling of combinational block 0 creates a single-pulse power wave within the time window between cycle 0 and cycle 1. Since the resolution requirement of most on-chip power meters is greater than one cycle, we calculate the average power per cycle. We denote the average dynamic power created by combinational block 0

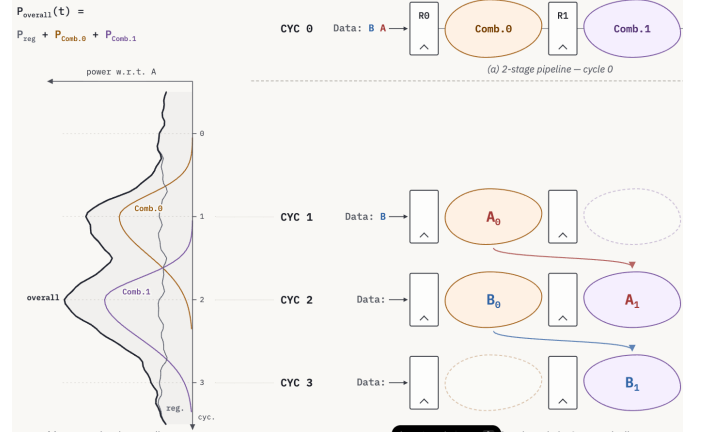


Fig. 3. Pipeline stages and the dynamic power of the sequential and combinational parts of each module.

as $P_{comb,0}(0 \rightarrow A_0)$, where $0 \rightarrow A_0$ means the data at the input port of combinational block 0 goes from 0 to A_0 in this cycle. In cycle 2, data B enters combinational block 0 and creates power $P_{comb,0}(A_0 \rightarrow B_0)$; meanwhile, combinational block 1's toggling creates power $P_{comb,1}(0 \rightarrow A_1)$. We use A_0 and A_1 to denote that these two data items are different, since A_1 is the output of combinational block 0 given A_0 . The total dynamic power created by the combinational blocks is

$$\text{Cycle 2: } P_{comb} = P_{comb,0}(A_0 \rightarrow B_0) + P_{comb,1}(0 \rightarrow A_1), \quad (3)$$

$$\text{Cycle 3: } P_{comb} = P_{comb,0}(B_0 \rightarrow 0) + P_{comb,1}(A_1 \rightarrow B_1). \quad (4)$$

As the registers are not clock-gated, the dynamic power of the registers is present in all three cycles.

From the above analysis, we can derive the power fluctuation curve shown in Fig. 3. Two conclusions are drawn and used as the foundation of our white-box power modeling:

- **Conclusion 1.** The power fluctuation of a module depends only on its input payload and the logic function of its combinational circuit. The power fluctuations of modules in different pipeline stages are independent.
- **Conclusion 2.** The average power of the circuit operating in one cycle is the summation of the average power of the modules in the different pipeline stages.

B. Modeling Methodology

Based on the above conclusions, the Generalized Additive Model (GAM) [32] is one of the best statistical approaches for building a cycle-accurate on-chip power model. As an extension of generalized linear models, a GAM does not multiply each feature by a fixed weight; instead, it applies an independent, arbitrary function f_i to each individual feature x_i :

$$g(E(y | x)) = \beta_0 + f_1(x_1) + f_2(x_2) + \dots + f_p(x_p). \quad (5)$$

In our power modeling problem, $E(y | x)$ is the expected power per cycle y given a set of input stimuli x . Each f_i represents a module's logic function. The link function $g(\cdot)$ indicates that the constructed functions f_i bridge the logic function's toggle activity to the average dynamic power it

generates per cycle. The fixed β_0 can be used to model the invariant power during the chip's operation, including static power and the always-on registers' dynamic power. Each x_i is a power proxy implemented from on-chip signals (nets).

X-OPM approximates f_i with simple operators that can be implemented in hardware with low overhead. For the white-box model, we degrade the GAM to a linear model to reduce modeling complexity:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p. \quad (6)$$

Two principles are proposed to build the model; we call them the *module-based method* and the *function-based method*. For the module-based method, x_i represents the activity of a module in one cycle, and β_i represents the average power of this module generated by this toggling activity. The activity of a module is approximately represented by the on-chip signals constructed following the four-type power proxy conventions:

- If we use the one-bit clock gating signal as x_i , this signal equals 1 when the clock of this module is open. No matter whether this module executes in a high-, middle-, or low-power scenario, the term $\beta_i x_i$ always equals β_i . Thus, β_i will ideally be trained to represent the average operating power of this module, provided the training test cases cover the full spectrum of this module's operating power.
- If we use the data gating signal as x_i , this signal equals 1 only when a certain part of the combinational block is activated. Thus, the power of this module equals β_i only when this part of the circuitry is activated, allowing more fine-grained modeling.
- The control payload controls the module to operate under a certain configuration. In the aforementioned example, when *mask* equals 0, Module A operates as a multiplier; when *mask* equals 1, Module A operates as multiply-and-add. This selective behavior can be modeled as

$$P = \beta_{\text{mult}} \cdot (\text{mask}=0) + \beta_{\text{mult-add}} \cdot (\text{mask}=1), \quad (7)$$

where β_{mult} represents the multiplier's power and $\beta_{\text{mult-add}}$ represents the multiply-and-add's power. If some modeling error is acceptable, one term can be used instead of two: we construct a new 2-bit signal $\text{mode} = \text{mask} + 1$ and construct the power model as

$$P = \beta_{\text{binned}} \cdot \text{mode}. \quad (8)$$

When $\text{mode} = 1$, $P = \beta_{\text{binned}}$; when $\text{mode} = 2$, $P = 2\beta_{\text{binned}}$. This technique is called *binning*; the modeling error comes from the fact that we forced the multiply-and-add's power to be twice the power of a single multiplier.

- The data payload, such as a data bus, normally contains tens to thousands of bits, and it would be expensive to pair each bit with a β_i weight. Binning can again be used to compress the number of signal bits. The finest-grained binning is to sum the bits to compute the Hamming weight of the data bus as the power proxy. For example, for a 4096-bit data bus `data[4095:0]`, a pipelined adder tree sums it into a 12-bit signal:

$$\text{data_sum} = \sum_{i=0}^{4095} \text{data}[i], \quad (9)$$

which is paired with a single β_i , reducing the proxy from 4096 bits to 12 bits. A more coarse-grained way is to use a multiplexer; for example, a 2-bit `data_bin` signal can be created as `data_bin = (data_sum ≥ 2048) ? 1 : 0`. Unlike the control payload, each bit of the data payload normally affects the power in a more homogeneous way, so more aggressive binning can be applied to the data payload than to the control payload.

A function-based method build the power model from the functional perspective instead of the physical perspective. The x_i represents whether a function is being executed or not, and the β_i represents the average operating power of this function. This method is particularly useful when you want to build the power model only from the control part of a large module. Like the tensor core typically contains repetitive MAC (multiply-and-accumulate) modules. If you build power-model for each MAC, you need to instantiate one power model per MAC, which incurs area and power overhead by implement repetitive power modules. **[TODO: consider illustrate it with a graph]**. A smater way is to build the power model directly from the controller of tensor core. For example the following three proxies can be used:

1. Type-A: Busy signal indicating that tensor core is executing the Matmul (matrix multiplication) instruction using its MAC units.
2. Type-C: Data type signal indicating which data type, such as `int8`, `bfloat16`, is being executed by the tensor core for the current instruction.
3. Type-D: Data signal indicating the data toggle rate of the input data bus of the tensor core. This signal also reflects the MAC utilization rate of the tensor core.

Note that these three proxies clearly have feature interactions. The MAC utilization rate under different data type may not affect the power with the same factor. Similar to eq.7, we could build our interacted features like this

$$P = \beta_{\text{dtype0}} \cdot (\text{dtype} == 0) + \beta_{\text{dtype1}} \cdot (\text{dtype} == 1) \quad (10)$$

Another point worth noticing is that high-power cases normally occupy only a small portion of the training dataset, for two reasons. (1) Activating high power requires a very specific stimulus that activates many combinational circuits, or some specific high-power circuits, at the same time, together with a high data toggle rate; such cases are harder to construct than the massive number of low-power cases. (2) Simulating a high-power case is much slower than simulating a low-power case, because the overall toggle rate is higher and a commercial simulator such as PTPX needs more time to traverse the netlist and calculate the resulting power.

When training an ordinary linear model, Gaussian elimination fits β_i to represent the average over all training samples fed to the optimizer. This general property remains unchanged when using regularized training methods such as Ridge or Lasso. Thus, when using a clock gating signal as x_i , one normally models middle-to-low power, because the training set covers all operation scenarios and most of them are not high-power cases. Conversely, when using a data gating signal as x_i , one normally models high-power cases: if the circuit

designer implemented data gating, it usually means that this part of the combinational circuit is high-power and deserves an extra AND gate to gate its power; when that part of the circuit operates (data gate deasserted), it is normally a high-power scenario.

The model accuracy of the white-box method is proportional to the modeling effort. At the coarsest granularity, we can use the clock gating signal of the top module of the circuit to be modeled, obtaining only the average power of this circuit when executing a wide spectrum of benchmarks. To increase accuracy, we model all the sub-module powers and add them together. If some sub-modules are still not accurate enough, we continue splitting them until a satisfying model accuracy is achieved. Note also that the modeling effort of using proxy types C and D is normally much higher than using types A and B, so they should be used only to model the fine-grained combinational power of a module.

C. Dataset for Model Fitting

Building the on-chip power meter normally requires collecting the training, validation and testing dataset via simulating the power trace of test cases. The quality of the test cases affects the model quality drastically. The guideline of building test cases are described below.

An ideal linear model has all its n features (proxies) orthogonal to each other. If a dataset contains n samples (test cases), where each sample has exactly one non-zero feature (one test cases only see one proxy toggles). Then the linear equation (6) can be solved deterministically

$$\beta = (X^T X)^{-1} X^T y$$

and each coefficient (β) represents the power of that test case. If multiple samples with different power value (y) and proxy toggle (x_i) are used in the training dataset, then β_i represents the average power of all test cases used to train proxy- i . This summarizes a way of building test cases to train an ideal linear model. However, building multiple test cases for each proxy significantly increases the modeling effort. Thus, we propose to reuse the **functional test cases** as the power training cases.

The functional test cases are built for verifying the functional correctness of the RTL during the design phases. Thanks to the high tape-out NRE (Non-Recurring Engineering) cost, production-level RTL code is usually well-tested before tape-out to minimize the chance of finding post-silicon bugs. Hierarchical functional tests, including the unit test, integration test and emulation, are normally built to maximize the code coverage rate and the functional coverage rate of the RTL design. A line of RTL code or a hardware function required by the chip spec. is marked as "covered" only if they have been hit during the verification phase. It is common that the code and functional coverage rate should reach over 90% and 100%, respectively, for high-quality tape-out. Since the functional test cases usually covers most of the code and all the functions of the design, the functional test cases can be regarded as good approximation towards the aforementioned ideal test cases.

Several types of functional cases are commonly built before tape-out:

- **Direct cases:** cover all the power proxies proposed by the designer, balance the dataset, and support regression. They cover the basic power scenarios, such as voltage droop and TDP, and are not related to real scenarios.
- **Random cases:** randomize the test parameters of sanity cases. Normally runs 24/7 before and after tape-out to look for uncovered corner cases.
- **Benchmark cases:** real scenario test cases, such as a full neural network layer. Normally very long and needs to be run on the emulator.

Our model is trained with the direct and random cases in order to cover most of the design features. Benchmark cases are used as testing dataset in order to see the performance of running model on the real-world scenarios. Additional direct cases need to be built if the functional test cases fail to cover the full range of the module's dynamic power. The following two criteria are used to check the coverage of the dataset:

- 1) the power of the dataset ranges from the module's static power to the module's maximum power.
- 2) all the power proxies in the final power model have toggled in at least one test case.

IV. AUTOMATIC MODEL BUILDING PROCEDURE

Ideally, one could build the entire power model using the white-box methodology introduced above. In reality, however, the modeling effort is prohibitive for a large, complex design such as a processor. Worse, the white-box methodology requires progressively more proxies as one tries to model the middle-to-low power range accurately, because capturing small power fluctuations demands the inclusion of more dedicated hardware features. In this section, we therefore propose an algorithm that builds the power model automatically while remaining faithful to the X-OPM modeling methodology.

A. Characteristic of the Power Modeling Problem

Automatic model building is closely related to black-box modeling [28]: a large set of candidate proxies—for example, the complete set of I/O signals or internal nets of a module—is fed to a statistical regressor, which is fitted to training data consisting of simulated power traces of various benchmark scenarios. Prior arts have mainly tried linear models, tree-based models, and neural networks, without an in-depth discussion of why a particular model family was chosen. From a statistical point of view, these model families fit regression problems with different characteristics:

- **Linear:** If the regression problem is dominated by a few variables whose relationships with the target are smooth, almost any statistical model works. Simple models such as polynomials or splines are usually good enough even when the variables have different distributions; if the variables are linear and i.i.d. (Independent and Identically Distributed), a plain linear model may suffice.
- **Tree:** If a few variables dominate but their relationships are irregular—with steps and sudden jumps—tree-based models fit better, because their piecewise-constant splits directly capture non-smooth, discontinuous relationships

that smooth function classes such as linear or polynomial models cannot represent well.

- **Neural Network:** If the regression problem must be fitted with a large number of weak variables (each individually having low correlation with the target), whose interactions or combinations are complex but smooth, and the dataset contains a large quantity of samples, then neural networks are the best choice.

We emphasize that choosing a model that matches the characteristics of the regression problem significantly affects the accuracy and stability of the resulting power model. For example, tree-based models are known to outperform neural networks when fitting tabular data [34].

The original dataset we collect is a 2D table: each column represents a power proxy (an RTL net), and each row is a sample recording the binary values of the proxies in one simulation cycle. As discussed for the toy design and the four proxy types in Section III, the interactions between RTL nets are typically non-smooth (e.g., conditional relationships). The original dataset can contain a very large number of features—possibly exceeding the number of samples—since the total number of RTL nets is huge; yet only a few strong signals among them contribute to the regression, per the earlier analysis of the hardware toy model. In addition, accurate power simulation with PTPX or emulation is slow, so only a small dataset can typically be collected in the limited window—usually several months—between the start of RTL development and tape-out. In summary, the dataset has the following characteristics:

- A few strong features dominate the regression problem.
- Non-smooth relationships pervasively exist between features.
- The training dataset is small.

Given these characteristics, we propose to construct a *derived* dataset from the original one that contains far fewer features (normally within a few hundred) on which the regression is performed. Since this reduction procedure must be hardened on-chip, it cannot be computationally heavy. The relationships among the derived features may or may not be smooth, so we try several lightweight models and select the one that fits best. Model construction proceeds top-down: each sub-module may be fitted by a standalone model with a different backbone, and the final power model is a GAM that sums the outputs of these distributed models. Our approach is separated into three steps—feature selection, feature interaction, and model regression—each performed automatically with the algorithms elaborated below.

B. Feature Selection

Feature selection is a coarse-grained procedure to quickly prune out the proxies that are uncorrelated to power. It has been well-utilized in the prior black-box modeling works such as APOLLO [28] and DEEP [29].

When modeling a module, using ports and nets as the origin of proxies is a commonly adopted approach. We also choose to use either input ports or the internal nets, but with different reason. Existing works [cite GRANNITE][cite

Simmani] [28] [29] reasons that nets are the physical cause of dynamic power, and ports are what you can cheaply observe. Our reasoning is based on the design model we described in Fig.2.: Input ports usually contain the most abundant information about a module, including the flow control signals (Fig.2.), the clock gating signals and the control and data payload. The toggling of input ports must cover all the functions a module could perform. This also makes input ports the fundamental interface for receiving test stimuli in various kinds of chip test environment. However, some control payloads on input ports can be encoded into busses. For example, using decimal (1, 2, 4) instead of one hot (001, 010, 100) to represent control payload normally saves more bit-width. This encoding creates complexity for the feature interaction procedure since the algorithm needs to learn the relationship between an encoded port to the power. That is why we also add internal nets to the origin of proxies. As for the output ports, since their toggling combinations are the result of the toggling of input ports, we do not chose them to avoid adding repetitive information. The effectiveness of such a proxy selection strategy will be discussed in the experiment session.

Features with zero variance on the training set will be removed first, as constant signals carry no predictive information. Then, we implemented five feature selection algorithms, executing them in parallel to select the top-performing one for each module. To manage resource constraints during model training, we integrated a timeout mechanism to terminate computationally infeasible tasks. The algorithms are listed below, ranked from lowest to highest computational complexity.

- **Variance Threshold** algorithm is a simple unsupervised method that removes all features whose variance does not meet a specified threshold.
- **Pearson Coefficient** algorithm ranks proxies based on the magnitude of their Pearson coefficient to the target, then drops the proxies that have coefficient lower than a threshold.
- **Model-based** algorithm uses a supervised machine learning estimator to measure the predictive power of each variable in a dataset. It drops features that do not contribute to the model's accuracy in one step based on metrics like tree-based importance.
- **DEEP** [29] algorithm implements MCP (Minimax Concave Penalty) regression used by APOLLO [28] to quickly prune out uncorrelated features, follows by an optional orthogonal matching pursuit greedily prunes the number of proxies to K .
- **RFE** (Recursive Feature Elimination) is a supervised feature selection algorithm that trains a model, evaluates feature importance, and prunes the least impactful features in a repeating loop. This cycle continues until the data is trimmed down to a pre-set number of top-performing variables.

C. Feature Interaction

Feature interaction is rarely mentioned explicitly in prior black-box OPM flows. APOLLO and DEEP select a sparse proxy set and fit a linear model directly; however, selection

only removes proxies with weak marginal association to power and does *not* guarantee that the retained proxies are linearly related to dynamic power. As Section III shows, type-A/B/D proxies often contribute multiplicatively (clock gate $\times$ busy $\times$ data toggle), while type-C proxies act selectively (different data types scale MAC power differently). Fitting a plain linear model on the selected set therefore leaves residual non-linearity that appears as unstable accuracy across benchmarks. The feature-interaction stage closes this gap and is the algorithmic counterpart of the manual interaction construction used in white-box X-OPM.

Feature interaction is rarely mentioned explicitly in the prior black-box modeling literature. For example, APOLLO [28] fits the proxies selected by MCP algorithm directly with a linear regressor. Since proxies selected by MCP only screens out features that have low correlation with the power, but do not guarantee to be linear w.r.t. power and i.i.d, APOLLO does not always yield the best result in our experiment.

We propose to add an extra feature interaction stage after the feature selection stage to transform the non-linear proxies before fitting them with the regressor. Since the feature interaction algorithm and the regressor needs to be implement on-chip, we want to use light-weight algorithms for feature interaction and linear model for the regressor. Given the characteristic of the power modeling problem discussed in section IV.A, we choose to use **GBDT (Gradient Boost Decision Tree)**, a tree-based algorithm, to perform feature interaction on the selected proxy set.

Tree-based algorithm bridges the automatic modeling procedure to the X-OPM methodology. In section III.A, when building power model in the white-box fashion, the multiplicative and selective relationship between the proxies need to be discovered and encoded manually. In contrast, GBDT is capable of automatically build these relationship. Recall Figure 2, the most important multiplicative relationship in the VLSI circuit is multiplying the single-bit flow control signal, clock or data gating signal with the payload signals. This is reduced to the *AND* relationship in circuit design. Decision tree discover such relationships by adding a selective *AND* condition to the proxy set every time when it branches, makes it one of the best candidate for our modeling problem. Our subsequent experiments show that using tree-based model to perform feature interaction performs better than MLP-based or attention-based [35] deep model.

D. Model Fitting

The training, testing and the validation dataset are standardized with z-score.

The power model of each circuit module is a two-stage composition of a GBDT follows by a linear model, trained end to end. The input to the linear model includes the proxies selected by the feature selection algorithm and the proxies derived by GBDT. Elastic Net is chosen to be our linear model. It combines L1 (Lasso) and L2 (Ridge) regularization to balance feature selection and model stability. The L1 penalty encourages sparsity by driving coefficients of less predictive features to exactly zero, while the L2 penalty manages multicollinearity by shrinking coefficients of correlated features

proportionally. By combining them, the L2 penalty enforces a grouping effect, ensuring that highly correlated variables are retained together rather than having the L1 penalty arbitrarily select only one. The loss function is given by:

$$Loss = WRSS + \lambda \left(\alpha \sum_{j=1}^p |\beta_j| + \frac{1-\alpha}{2} \sum_{j=1}^p \beta_j^2 \right) \quad (11)$$

The ElasticNet is fitted with weighted least squares (WLS) rather than ordinary least squares. The $WRSS$ above denotes the weighted residual sum of squares:

$$WRSS = \sum_{i=1}^n w_i (y_i - \hat{y}_i)^2 \quad (12)$$

$WRSS$ assigns each training sample i a pre-determined non-negative weight w_i . The prediction accuracy is biased towards the samples with larger w_i , which is particularly useful in certain cases. For example, when building OPM to predict processor's peak power, WLS can improve the prediction accuracy of the high-power cases by sacrificing the accuracy of the low-power cases.

When fitting the ElasticNet, we impose two structural constraints. First, the intercept is fixed to be the static power of the module being modeled. When all proxies are zero meaning the module is idle, the predicted power should equal to its static power. Second, all coefficients are constrained to be non-negative, meaning that the dynamic power of a sub-module or a function can never be negative. These two constraints force the coefficients to represent the dynamic power as designed in Section III.B.

The model's hyperparameters are automatically tuned using Bayesian optimization with the Tree-structured Parzen Estimator (TPE). In each trial, a new configuration is selected and trained on the training dataset. The validation dataset is used to enable early stopping. Each configuration is then evaluated based on its R^2 (coefficient of determination) using the validation data. This search process stops after a fixed time limit, and the best-performing configuration is used to retrain the final model. For the GBDT, the tuning process optimizes the number of trees, tree depth, learning rate (evaluated on a logarithmic scale), and the row and column subsampling ratios; For the ElasticNet, the search optimizes the regularization strength as well as the ℓ_1/ℓ_2 mixing ratio.

V. FRAMEWORK OF BUILDING OPM

A. OPM Fitting Procedure

The OPM building procedure is shown in Fig. 4 and comprises a training-set generation stage and a predictor construction stage. In the training-set generation stage, the design is first synthesized and layout to generate the netlist, SDF, and SPEF files. Then, behavior simulation is applied to obtain the activity trace. Time-based power simulation then generates a golden power trace. Finally, the activity and power traces are aligned and assembled into the raw dataset. Since the input to the OPM is RTL signals obtained at design time, the dataset deliberately pairs RTL-level features with gate-level power labels.

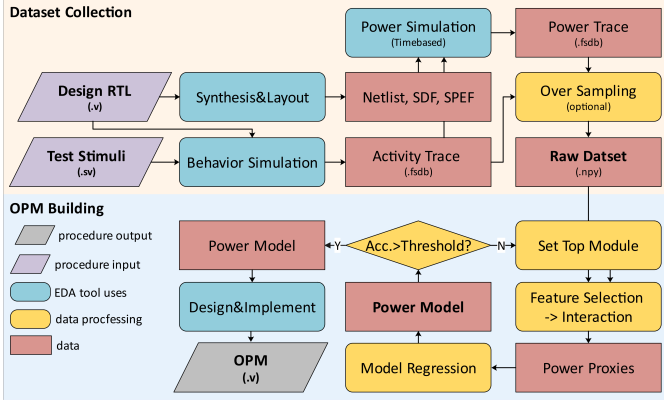


Fig. 4. The automatic X-OPM modeling flow

One worth-noticing problem is that the collected dataset is usually highly imbalanced – it contains far fewer sample of the high-power cases than the middle-to-low-power cases. This is a natural phenomena because constructing high-power cases require a large quantity of modules to toggle at the same time. However, fitting such imbalanced dataset with least squares biases the model towards the middle-to-low-power cases. This is because the least squares loss function (Eq. (12)) minimizes the sum of squared residuals, which is dominated by the abundant middle-to-low-power cases. To solve this problem, we apply ADASYN, an oversampling technique, to generate the synthetic samples for the high-power cases, generating synthetic samples adaptively where the data is sparse.

In the OPM's building stage, we select a top module to be modeled and apply feature selection and feature interaction to obtain its power proxies. Model regression on these proxies produces a candidate power model, which is evaluated on the held-out test set using R^2 , RMSE and sMAPE. If the model fails to reach the target R^2_{target} , we descend one hierarchy level and repeat the procedure on its sub-modules, summing the resulting models as in Eq. (5). Human intervention, including the add and remove the proxies constructed during the automatic process, are often applied to further improve the quality of the model. When a sub-module consumes less than 5% of its parent's power, it is usually not modeled and its power is absorbed into the intercept of the linear model.

B. OPM Deployment Procedure

After we fix the coefficient and the proxy of OPM in floating point number, we need to build the hardware of the OPM based on the fix-point arithmetic.

The linear model with pairwise interaction terms is implemented in hardware as

$$\text{pwr} = \left(\sum_{i=0}^{15} \alpha_i x_i + \sum_{j=0}^7 \beta_j x'_j x''_j + \gamma \right) \gg dp, \quad (13)$$

where $x'_j, x''_j \in \{x_0, x_1, \dots, x_{15}\}$, and the final right-shift by dp implements the online quantization (shift-and-add). Fig. 5 shows the architecture of the toggle detector that collects the proxy signals and evaluates the model. [TODO: Complete this subsection: parameterized model hardware design details.]

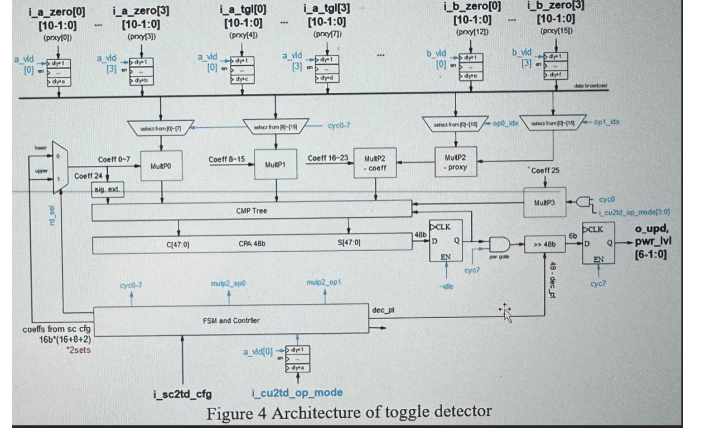


Fig. 5. Architecture of the toggle detector and the model evaluation datapath.

VI. EXPERIMENTS

A. Experiment Setup

We demonstrate the power modeling results on an open-sourced processor, the XuanTie C906 [36]. The C906 is a 12-stage out-of-order edge processor developed by T-Head, Alibaba. It supports the RVV64 instruction extension with an integrated 128-bit vector lane, making it capable of running edge machine learning workloads with up to 16 GOPS of INT8 throughput. The C906 is silicon-proven and has been manufactured into end products; it is fully supported by T-Head's open-sourced neural network runtime library ShiHuLan, alongside its dedicated AI compiler HHB.

To evaluate the power, we first synthesize and lay out the C906 core with the TSMC 28 nm HPC+ technology node, then perform post-layout power simulation with SDF annotation to obtain the power traces. To achieve sign-off-quality simulation accuracy, we implement the design with industrial-standard commercial tools: synthesis with Synopsys Design Compiler 2025, layout with Cadence Innovus 21.1, and power simulation with Synopsys PrimePower 2025. The power simulation is conducted on an HPC cluster with hundreds of CPU cores and tens of TB RAMs.

B. Modeling Result of C906

The implemented core occupies about 0.87 mm² with a 42% overall cell utilization rate. Timing is closed at 1 GHz under 0.9 V VDD at the TT corner; SRAM and logic share one voltage rail, and a mixture of ULVT, LVT, and SVT cells is used to close timing. Under these conditions, the average dynamic power exceeds 103 mW when running the heavy CoreMark workload, with peak power over 159 mW, aligning with XuanTie's official report [36].

Fig. 6 shows the detailed power breakdown of the C906 running the CoreMark benchmark. EEMBC's CoreMark is an industrial-standard benchmark that measures the performance of a CPU or MCU; it includes a series of kernels such as matrix multiplication, linked-list processing, and state machines. Each bar in the figure shows the power of a sub-module of the C906 core: the bar height represents the module's average

power, and the error bar represents the range of power fluctuation during execution. The C906 core top module is labeled `x_aq_core`. We call the direct sub-modules of `x_aq_core` the level-1 sub-modules, and the sub-modules under level-1 the level-2 sub-modules; modules whose names end with the suffix `_top` are level-1 sub-modules, and the other modules are level-2 sub-modules.

Our modeling strategy is to model the level-1 sub-modules using the X-OPM method; if a level-1 model does not converge, we descend to its level-2 sub-modules. If a level-2 module's power is smaller than 10% of the level-1 module containing it, we consider it too small to model, skip it, and accept the resulting modeling error; the power of such a module is absorbed into the constant coefficient of the level-1 model during training.

In the feature-selection experiment, we use the same budget, $K = 20$, for every method and pair each selected set with the same five downstream backbones: ElasticNet, GBDT, RuleFit, MLP, and FT-Transformer. We repeat this comparison for both input-port and internal-net candidates of each modeled C906 module. Holding the feature budget, data split, and downstream models fixed isolates the effect of the selector; the final configuration is chosen by validation accuracy rather than by the selector's training score. This comparison is necessary because no selector is uniformly optimal: correlation filters favor strong marginal proxies, sparse linear methods favor jointly linear proxies, and wrapper methods may retain complementary proxies at substantially higher training cost.

Discovered interactions are reported among the top-20 features by model importance after selection. RuleFit interactions are quantified by weighted rule co-occurrence (pairs that share a nonzero rule contribute $|\text{coef}| \times \text{support}$). GBDT interactions use mean absolute SHAP interaction values on a subsample of training rows; Friedman's H -statistic and partial dependence plots remain useful classical diagnostics for the same tree models [37]. FT-Transformer interactions are read from the averaged feature–feature attention matrix (CLS token excluded). Linear ElasticNet and plain MLP backbones do not expose an explicit interaction matrix; they serve as ablation baselines in the experimental grid.

C. Compare X-OPM with MLP and Transformer

Fig.7

When many weak proxies must mix smoothly rather than through hard conjunctions, we use the Feature Tokenizer Transformer (FT-Transformer) [35]. Each selected proxy is embedded as a token; stacked multi-head self-attention blocks let every proxy attend to every other proxy, producing a data-dependent soft interaction graph. Averaging attention weights over layers, heads, and samples yields a feature–feature attention matrix that is directly interpretable as a soft counterpart of RuleFit's hard co-occurrence structure. A lightweight prediction head then maps the attended representation to power.

We report the model indicators and metrics against the following baselines: plain OLS, WLS, and Ridge; black-box methods APOLLO [28] and DEEP [29]; tree-based and ensemble-tree-based models; and neural network models. We

plot the fitting results and the residual maps of the training and testing sets. [TODO: results]

We further demonstrate the effectiveness of lower-level (sub-module) modeling using RuleFit and GBDT, and show the convergence of the black-box methodology toward the white-box methodology. [TODO: results]

An ablation study proves the effectiveness of the feature interaction step: without it—i.e., using plain GBDT, MLP, and ElasticNet directly on the selected features—the accuracy degrades. [TODO: results]

D. Hardware Overhead Evaluation

Finally, we compare the area of using Transformer, MLP, GBDT, and RuleFit backbones to construct the final GAM model, and evaluate the hardware overhead of each component of the on-chip power meter—the feature selection logic, tree-based models, MLPs, linear models, and GBDTs—implemented under the TSMC 28 nm HPC+ process node. [TODO: results]

E. Selection of Original Features

We first compare using input signals, internal nets, and output signals of a module as the proxy source. Using outputs performs poorly; between inputs and internal nets, sometimes inputs win and sometimes internal nets win, sometimes the combination of both wins, so in practice it is better to try both and select the better one.

Using inputs and nets, Fig. 7 shows the feature importance of the top-20 features for the MLP and GBDT models: the Transformer and MLP models need a larger number of features to be precise, with smooth rather than abrupt importance distributions, whereas the GBDT importance is concentrated in a few strong features. For feature selection, we perform a parameter search on the number of features: starting from an initial value (e.g., $n_{\text{feature}} = 32$ per module), we iteratively increase or decrease it to maximize R^2/n_{features} . [TODO: results]

VII. CONCLUSION

[TODO: Write conclusion: X-OPM provides a fully explainable, automated OPM modeling methodology reaching MLP-level accuracy at a fraction of the hardware cost; the methodology is packaged into a platform applicable to any digital circuit.]

REFERENCES

- [1] J. Hines *et al.*, “Empirical measurements of AI training power demand on a GPU-accelerated node,” arXiv preprint, Tech. Rep., Dec. 2024.
- [2] Y. Wang *et al.*, “TokenPowerBench: Benchmarking the power consumption of LLM inference,” arXiv preprint, Tech. Rep., Dec. 2025.
- [3] E. Grochowski, D. Ayers, and V. Tiwari, “Microarchitectural simulation and control of di/dt-induced power supply voltage variation,” in *Proc. Int. Symp. on High-Performance Computer Architecture (HPCA)*, Feb. 2002.
- [4] M. D. Powell and T. N. Vijaykumar, “Pipeline damping: A microarchitectural technique to reduce inductive noise in supply voltage,” in *Proc. Int. Symp. on Computer Architecture (ISCA)*, May 2002.

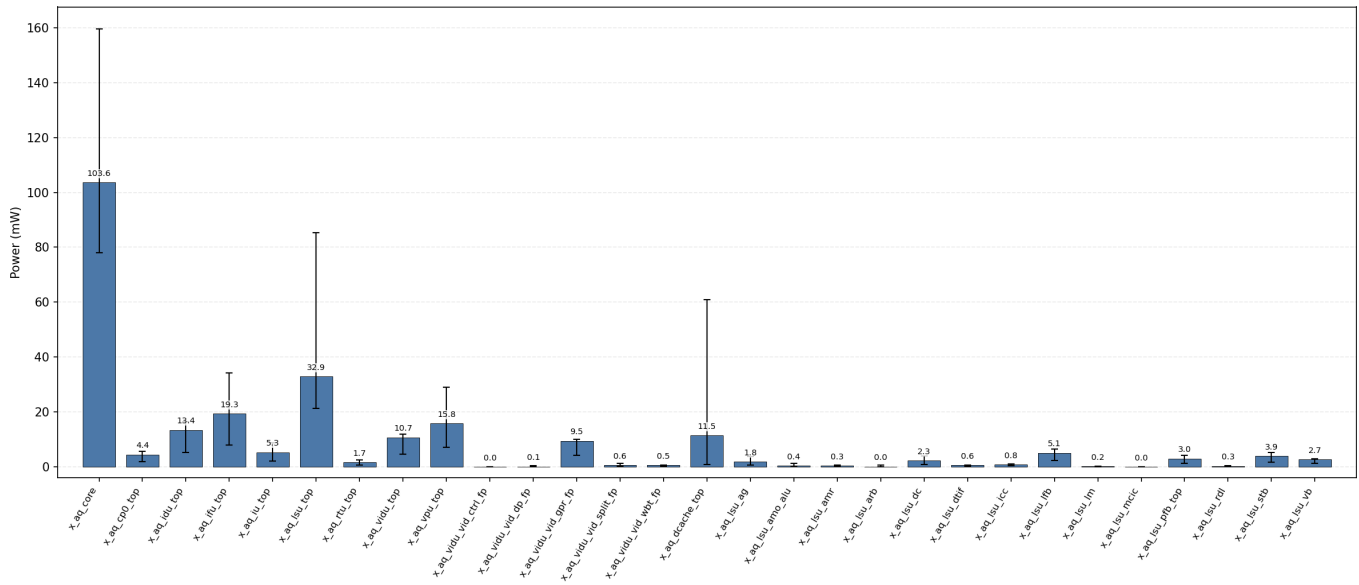


Fig. 6. Power breakdown of the C906 core running the CoreMark benchmark. Bar heights show average module power; error bars show the power fluctuation range during execution.

- [5] R. Joseph, D. Brooks, and M. Martonosi, "Control techniques to eliminate voltage emergencies in high performance processors," in *Proc. Int. Symp. on High-Performance Computer Architecture (HPCA)*, Feb. 2003.
- [6] J. Leng, Y. Zu, M. Rhu, M. Gupta, and V. J. Reddi, "GPUVolt: Modeling and characterizing voltage noise in GPU architectures," in *Proc. Int. Symp. on Low Power Electronics and Design (ISLPED)*, Jul. 2014.
- [7] J. Leng, Y. Zu, and V. J. Reddi, "GPU voltage noise: Characterization and hierarchical smoothing of spatial and temporal voltage noise interference," in *Proc. Int. Symp. on High-Performance Computer Architecture (HPCA)*, Feb. 2015.
- [8] R. Thomas, K. Barrera, J. Leng *et al.*, "EmerGPU: Understanding and mitigating resonance-induced voltage noise in GPU architectures," in *Proc. Int. Symp. on Performance Analysis of Systems and Software (ISPASS)*, May 2016.
- [9] I. Paul, S. Manne, M. Arora, W. L. Bircher, and S. Yalamanchili, "Cooperative boosting: Needy versus greedy power management," in *Proc. Int. Symp. on Computer Architecture (ISCA)*, Jun. 2013.
- [10] C. Berry *et al.*, "Proactive power management in IBM z15," *IBM Journal of Research and Development*, Aug. 2020.
- [11] S. Bang *et al.*, "A proactive system for voltage-droop mitigation in a 7-nm Hexagon processor," *IEEE Journal of Solid-State Circuits*, Feb. 2021.
- [12] C. Lefurgy *et al.*, "Adaptive clocking in the POWER9 processor for voltage droop protection," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2017.
- [13] C. Berry *et al.*, "IBM z14: Processor characterization and power management for high-reliability mainframe systems," *IEEE Journal of Solid-State Circuits*, Jan. 2019.
- [14] G. Ko *et al.*, "A 65-nm proactive power management technique with real-time machine learning engine for droop prediction and mitigation," *IEEE Journal of Solid-State Circuits*, Nov. 2024.
- [15] Y.-L. Chen *et al.*, "Run-time power management system by on-die power sensor with silicon machine learning-based calibration in a 3-nm mobile SoC," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2025.
- [16] J. Kim *et al.*, "Proactive power management-based supply regulation with online learning for variation-tolerant workload-aware processors," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2026.
- [17] S. Gupta and F. N. Najm, "Lookup table power macro-models for behavioral library components," in *Proc. IEEE Alessandro Volta Memorial Workshop on Low-Power Design*, Jan. 1999.
- [18] C. Isci and M. Martonosi, "Runtime power monitoring in high-end processors: Methodology and empirical data," in *Proc. IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, Dec. 2003.
- [19] S. Sen *et al.*, "Accurate fine-grained processor power proxies," in *Proc. IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, Dec. 2012.
- [20] Z. Lin, Z. Yuan *et al.*, "An ensemble learning approach for in-situ monitoring of FPGA dynamic power," in *Proc. Int. Conf. on Field-Programmable Technology (FPT)*, Sep. 2020.
- [21] S. Ahuja *et al.*, "A novel approach for high-level power modeling of sequential circuits using recurrent neural networks," *Proc. IEEE Int. Symp. on Circuits and Systems (ISCAS)*, May 2005.
- [22] S. Bhanja *et al.*, "Neural network macromodel for high-level power estimation of CMOS circuits," *Proc. Int. Conf. on VLSI Design*, Oct. 2005.
- [23] V. Chandra *et al.*, "Neural network based macromodels for high level power estimation," *Int. Journal of Computer Science and Engineering*, Jul. 2007.
- [24] H.-T. Cheng *et al.*, "Wide & deep learning for recommender systems," in *Proc. Workshop on Deep Learning for Recommender Systems*, Jun. 2016.
- [25] Y. Zhou, H. Ren, Y. Zhang, B. Keller, B. Khailany, and Z. Zhang, "PRIMAL: Power inference using machine learning," in *Proc. ACM/IEEE Design Automation Conf. (DAC)*, Jun. 2019.
- [26] J. Fang *et al.*, "PROPHET: Predictive on-chip power meter in hardware accelerator for DNN," in *Proc. ACM/IEEE Design Automation Conf. (DAC)*, Jul. 2023.
- [27] —, "Data-pattern-based predictive on-chip power meter in DNN accelerator," *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems*, Nov. 2024.
- [28] Z. Xie, X. Xu, M. Walker, J. Knebel, K. Palaniswamy, N. Hebert, J. Hu, H. Yang, Y. Chen, and S. Das, "APOLLO: An automated power modeling framework for runtime power introspection in high-volume commercial microprocessors," in *Proc. IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, Oct. 2021.
- [29] J. Zhai, Y. Cai, and B. Yu, "DEEP: Developing extremely efficient runtime on-chip power meters," in *Proc. IEEE/ACM Int. Conf. on Computer-Aided Design (ICCAD)*, Oct. 2022.
- [30] Q. Zhang, S. Li, G. Zhou, J. Pan, C.-C. Chen, Y. Chen, and Z. Xie, "PANDA: Architecture-level power evaluation by unifying analytical and machine learning solutions," in *Proc. IEEE/ACM Int. Conf. on Computer-Aided Design (ICCAD)*, Oct. 2023.
- [31] D. Kim, J. Z. Ahn, J. Bachrach, and K. Asanović, "Simmani: Runtime power modeling for arbitrary RTL with automatic signal selection," in *Proc. IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, Oct. 2019.
- [32] T. Hastie and R. Tibshirani, "Generalized additive models," *Statistical Science*, vol. 1, no. 3, pp. 297–310, 1986.
- [33] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, "McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures," in *Proc. IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, Dec. 2009.

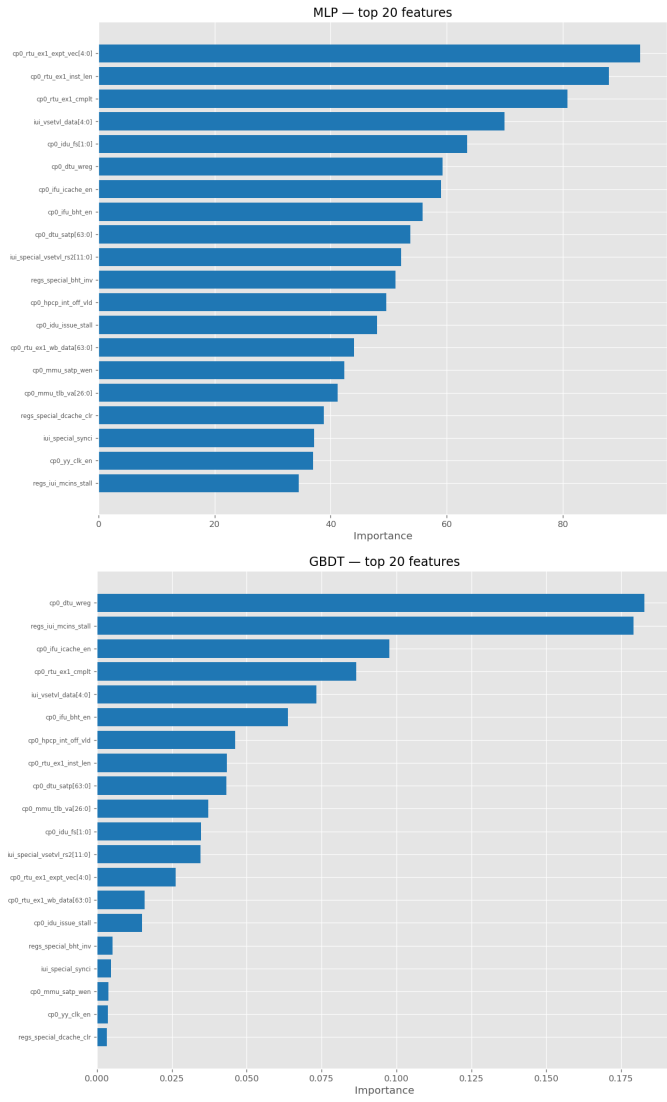


Fig. 7. Top-20 feature importance of the MLP model (top) and the GBDT model (bottom).

- [34] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on tabular data?” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, Jul. 2022.
- [35] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, “Revisiting deep learning models for tabular data,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [36] T-Head Semiconductor, “Xuantie c906 processor datasheet,” https://occ-oss-prod.oss-cn-hangzhou.aliyuncs.com/resource//1718100372780/XuanTie%20C906%20Processor%20Datasheet_20240607.pdf, 2024.
- [37] J. H. Friedman and B. E. Popescu, “Predictive learning via rule ensembles,” *The Annals of Applied Statistics*, vol. 2, no. 3, pp. 916–954, Nov. 2008.